

ADVANCED TECHNIQUES FOR ADVANCED DATABASES

SQL VIEWS

OBJECTIVES

- Understand the purpose of views.
- Discuss the advantages of using views in database management.
- Creating views and joining tables in views.

UNDERSTANDING VIEWS

- Views allows you to create custom virtual tables based on existing data.
- Virtual tables derived from one or more underlying base tables.
- Act as a layer between the users and the database, allowing users to interact with a subset of data or a customized representation of data.
- They provide a way to simplify complex queries, enforce data security, and present a more user-friendly interface to database users.

ADVANTAGES OF USING VIEWS

- Enhanced data security by restricting access to certain columns.
- Data abstraction to simplify queries.
- Improved performance by storing frequently used queries as views.

TYPES OF VIEWS

- Simple views – basic views that retrieve data from a single table
- Complex views – Views that involve multiple tables, joins, and filtering conditions.
- Materialized views – A special type of view that stores the query results physically for faster access.

CREATE VIEWS SYNTAX

CREATE VIEW View_Name AS SELECT statement

SIMPLE VIEWS

- Create a view called **customer_names** to retrieve specific columns from the **customers** table.

SIMPLE VIEWS

- **Creating a simple view**

```
CREATE VIEW customer_names AS  
SELECT customer_id, first_name, last_name  
FROM customers;
```

SIMPLE VIEWS

- **Creating a simple view**

```
CREATE VIEW customer_names AS  
SELECT customer_id, first_name, last_name  
FROM customers;
```

In this query we provide the users with a concise and relevant dataset from the CUSTOMERS table.

COMPLEX VIEWS

- Create a complex view called **order_details** that combines data from the **orders**, **customers**, **order_items**, and **products** tables.

COMPLEX VIEW

- **Creating a complex view**

```
CREATE VIEW order_details AS
```

```
SELECT
```

```
    o.order_id, o.order_date, c.first_name,  
    c.last_name, p.product_name, od.quantity
```

```
FROM orders o
```

```
JOIN customers c ON o.customer_id = c.customer_id
```

```
JOIN order_items od ON o.order_id = od.order_id
```

```
JOIN products p ON od.product_id = p.product_id
```

```
WHERE o.order_date >= '2024-01-01';
```

COMPLEX VIEWS

- By creating these views with joins, we can now retrieve comprehensive order details, including customer information, product names, and quantities, all in one convenient view.

HOW TO QUERY VIEWS?

- To query data from the created views use SELECT statement, similar to how you would query a regular table.

MATERIALIZED VIEWS

- Materialized views in PostgreSQL serve as a powerful feature for optimizing data retrieval, something that's particularly useful in scenarios requiring fast access to complex query results.
- To create materialized view, use the `CREATE MATERIALIZED VIEW` command, specifying whether to populate it with data immediately or to leave it empty for manual data loading later.
- This flexibility allows for efficient use of resources based on the specific needs of your application or analysis tasks.
- MySQL does not support materialized views.

MATERIALIZED VIEWS - SYNTAX

- CREATE MATERIALIZED VIEW view_name AS query WITH [NO] DATA;

MATERIALIZED VIEWS

```
CREATE MATERIALIZED VIEW order_details_mat AS
SELECT
    o.order_id, o.order_date, c.first_name,
    c.last_name, p.product_name, od.quantity
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
WHERE o.order_date >= '2024-01-01'
WITH NO DATA;
```

UPDATING DATA THROUGH VIEWS

- Updating a view in SQL means modifying the data it shows.
- Not all views can be updated instantly.
- **The UPDATE** keyword is used to update the view without changing the schema of the table. Update keyword will be used to update the values of the attributes.

UPDATING DATA THROUGH VIEWS

```
UPDATE hr_employee_view  
SET first_name='John Mark'  
WHERE emp_id=102;
```

INSERTING DATA THROUGH VIEWS

```
INSERT INTO hr_employee_view (emp_id, first_name, last_name)  
VALUES (101, 'Anna', 'Thompson');
```

DELETING DATA THROUGH VIEWS

```
DELETE FROM hr_employee_view  
WHERE emp_id=101;
```